

学习要点:

- 掌握算法的计算复杂性概念。
- 掌握算法渐近复杂性的数学表述。
- 掌握算法分析的基本法则。
- 掌握算法分析中常用的复杂函数。

算法渐近复杂性的定义：

设 $T(n)$ 是算法A的复杂性函数，当 $n \rightarrow \infty$ 时， $T(n) \rightarrow \infty$ 。

如果存在 $t(n)$ ，当 $n \rightarrow \infty$ 时，使得 $(T(n) - t(n)) / T(n) \rightarrow 0$ ，
称 $t(n)$ 是 $T(n)$ 的渐近性态，为算法A当 $n \rightarrow \infty$ 的渐近复杂性。

- 在数学上， $t(n)$ 是 $T(n)$ 的渐近表达式，通常是 $T(n)$ 略去低阶项留下的主项。它比 $T(n)$ 简单。
 - $T(n) = 3n^2 + 4n + 7$
 - $t(n) = 3n^2$

渐近分析的记号 $O, \Omega, \Theta, o, \omega$

在下面的讨论中，对所有 n ，函数 $f(n) \geq 0$ ， $g(n) \geq 0$ 。

(1) 渐近上界 O

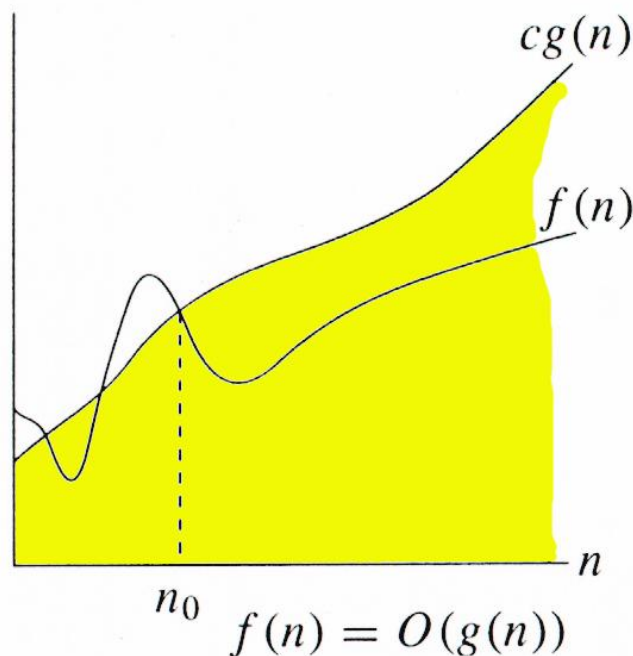
存在正常数 c 和 n_0 ，使得对所有 $n \geq n_0$ 有： $0 \leq f(n) \leq cg(n)$ ，

称 $g(n)$ 为 $f(n)$ 的渐近上界。记作 $f(n) = O(g(n))$

$$3n^3 = O(n^4)$$

$$2n + 3 = O(n^2) \quad \text{松散的界限}$$

$$2n + 3 = O(n) \quad \text{较好的界限}$$

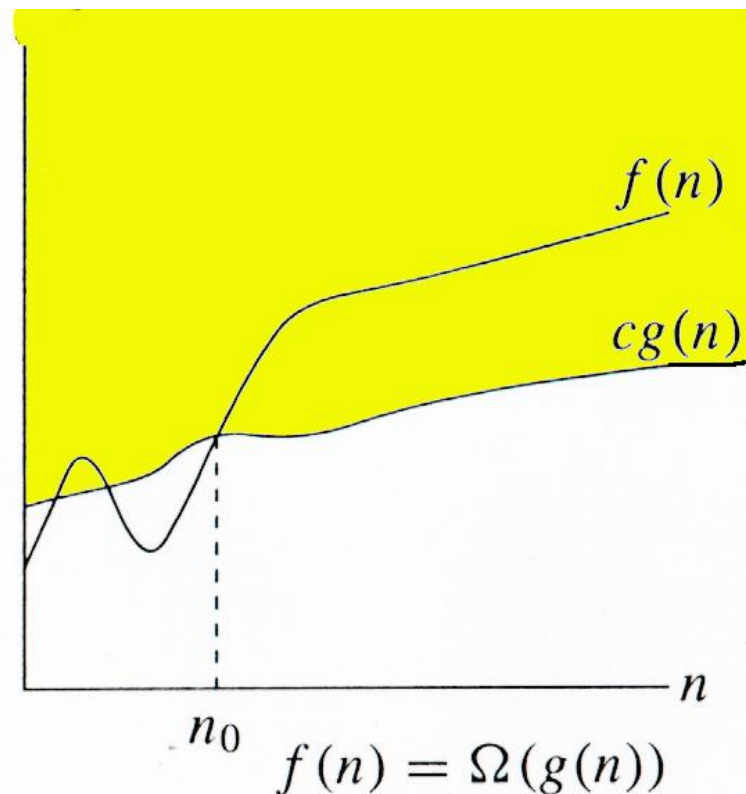


(2) 渐近下界 Ω

存在正常数 c 和 n_0 使得对所有 $n \geq n_0$ 有: $0 \leq cg(n) \leq f(n)$

，称 $g(n)$ 为 $f(n)$ 的渐近下界。记作 $f(n) = \Omega(g(n))$

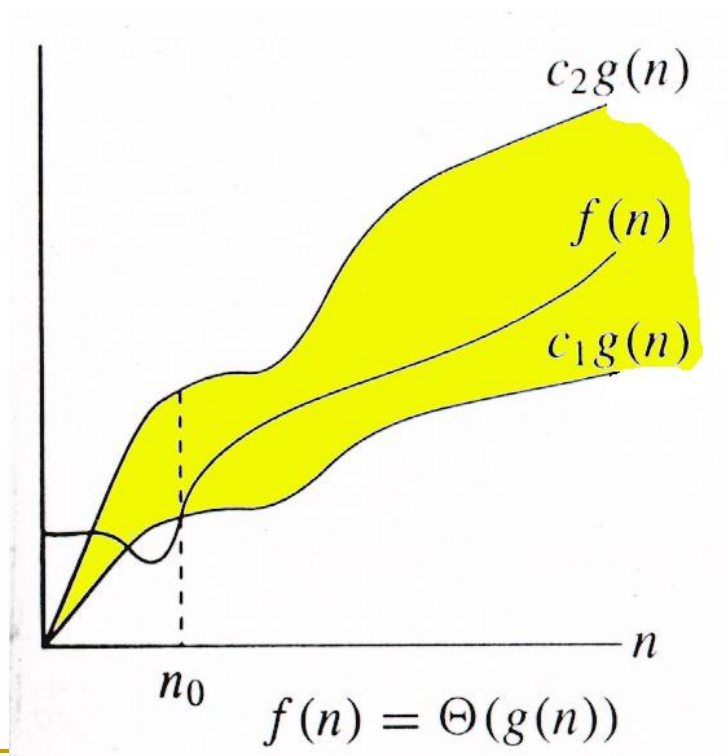
$$3n^3 = \Omega(n^2)$$



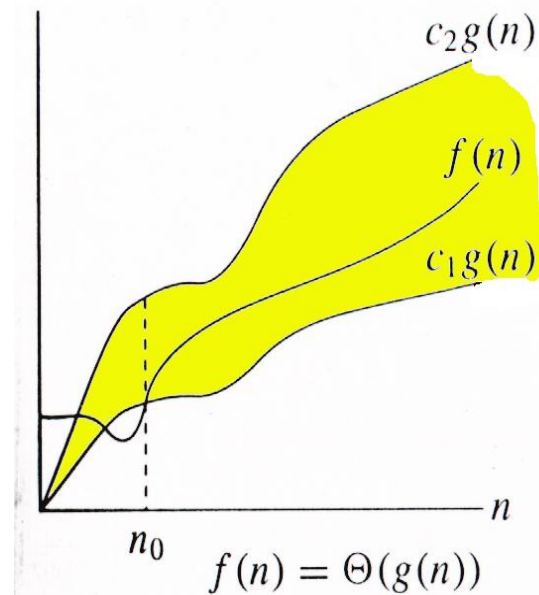
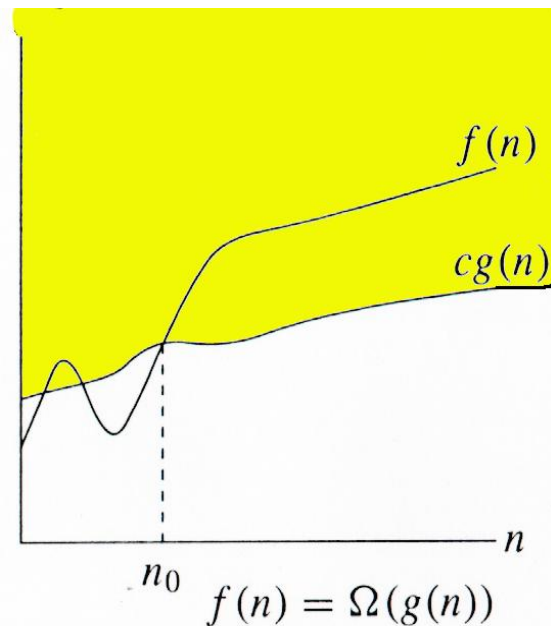
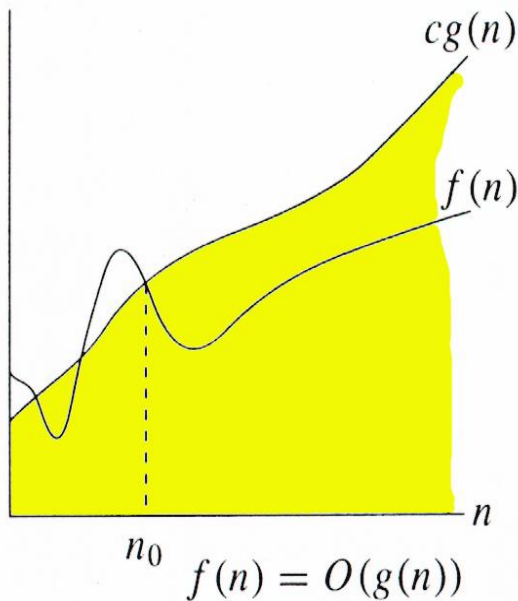
(3) 紧渐近界 Θ

存在正常数 c_1, c_2 和 n_0 使得对所有 $n \geq n_0$ 有: $c_1 g(n) \leq f(n) \leq c_2 g(n)$
，称 $g(n)$ 为 $f(n)$ 的紧渐近界。记作 $f(n) = \Theta(g(n))$

$$10n^2 - 3n = \Theta(n^2)$$



Θ, O, Ω 之间的关系



渐近上界 O

渐近下界 Ω

紧渐近界 Θ

Big-O, Θ , Ω 实例

略去低阶项和常数系数项，留下的主项

略去低阶项

- $4n + 5 \Rightarrow 4n$
- $0.5 n \log n - 2n + 7 \Rightarrow 0.5 n \log n$

略去常数系数项

- $4n \Rightarrow n$
- $0.5 n \log n \Rightarrow n \log n$
- $\log n^2 = 2 \log n \Rightarrow \log n$
- $\log_3 n = (\log_3 2) \log n \Rightarrow \log n$

(4) 非紧上界 $\circ$

对于任何正常数 $c > 0$, 存在正数 $n_0 > 0$, 使得对所有 $n \geq n_0$ 有: $0 \leq f(n) < cg(n)$, 称 $g(n)$ 为 $f(n)$ 的非紧上界。记作 $f(n) = \circ(g(n))$

(5) 非紧下界 ω

对于任何正常数 $c > 0$, 存在正数 $n_0 > 0$, 使得对所有 $n \geq n_0$ 有: $0 \leq cg(n) < f(n)$, 称 $g(n)$ 为 $f(n)$ 的非紧下界。记作 $f(n) = \omega(g(n))$

渐近分析中函数比较

紧渐近界 Θ $f(n) = \Theta(g(n)) \approx a = b;$

渐近上界 O $f(n) = O(g(n)) \approx a \leq b;$

渐近下界 Ω $f(n) = \Omega(g(n)) \approx a \geq b;$

非紧上界 o $f(n) = o(g(n)) \approx a < b;$

非紧下界 ω $f(n) = \omega(g(n)) \approx a > b.$

渐近分析记号的若干性质

(1) 传递性:

$$f(n) = \Theta(g(n)), \quad g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n));$$

$$f(n) = O(g(n)), \quad g(n) = O(h(n)) \Rightarrow f(n) = O(h(n));$$

$$f(n) = \Omega(g(n)), \quad g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n));$$

$$f(n) = o(g(n)), \quad g(n) = o(h(n)) \Rightarrow f(n) = o(h(n));$$

$$f(n) = \omega(g(n)), \quad g(n) = \omega(h(n)) \Rightarrow f(n) = \omega(h(n));$$

(2) 反身性:

$$f(n) = \Theta(f(n));$$

$$f(n) = O(f(n));$$

$$f(n) = \Omega(f(n)).$$

(3) 对称性:

$$f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n)).$$

(4) 互对称性:

$$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n));$$

$$f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$$

(5) 算术运算:

$$O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\}) ;$$

$$O(f(n)) + O(g(n)) = O(f(n) + g(n)) ;$$

$$O(f(n)) * O(g(n)) = O(f(n) * g(n)) ;$$

$$O(cf(n)) = O(f(n)) , \text{ } c \text{ 是一个正常数};$$

$$g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$$

有什么用?

由两个连续执行部分组成的算法

由两个连续执行部分组成的算法

$$O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$$

第一部分，应用某种已知的排序算法对数组排序；

第二部分，连续扫描该有序数组的元素，比较是否和指定元素相等

假设第一部分使用的排序算法的比较次数不会超过 $n(n-1)$ ，属于 $O(n^2)$

第二部分的比较次数不会超过 $n-1$ ，属于 $O(n)$

那么，**算法的整体效率**应该属于 $O(\max\{n^2, n\}) = O(n^2)$

学习要点:

- 掌握算法的计算复杂性概念。
- 掌握算法渐近复杂性的数学表述。
- 掌握**算法分析的基本法则**。
- 掌握算法分析中常用的复杂函数。

算法分析的基本法则

C++ 操作 常数时间， 记作 $O(1)$

CATEGORY	EXAMPLES	ASSOCIATIVITY
Operations on References	. []	Left to right
Unary	++ -- ! - (type)	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift (bitwise)	<< >>	Left to right
Relational	< <= > >= instanceof	Left to right
Equality	== !=	Left to right
Boolean (or bitwise) AND	&	Left to right
Boolean (or bitwise) XOR	^	Left to right
Boolean (or bitwise) OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= *= /= %= += -=	Right to left

算法分析的基本法则

非递归算法：

(1) 顺序语句

各语句计算时间相加；

(2) **if-else**语句

条件测试+分支计算时间的较大者；

(3) **for / while** 循环

循环体内计算时间*循环次数；

(4) 嵌套循环

循环体内计算时间***所有**循环次数。

递归函数

求解递归方程

嵌套循环

```
for i = 1 to n do  
  for j = 1 to n do  
    sum = sum + 1
```

$O(n^2)$

$$\sum_{i=1}^n \sum_{j=1}^n 1 = \sum_{i=1}^n n = n^2$$

嵌套循环

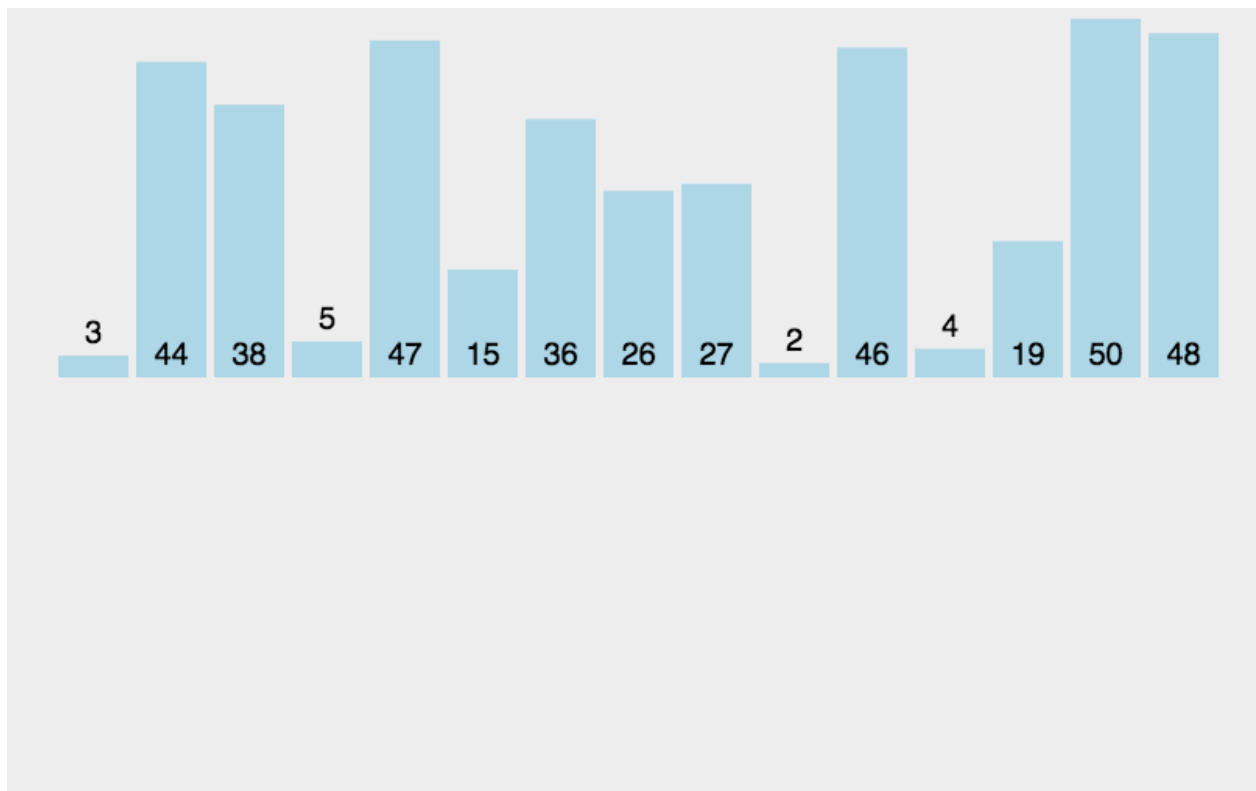
```
for i = 1 to n do  
  for j = i to n do  
    sum = sum + 1
```

$O(n^2)$

$$\sum_{i=1}^n \sum_{j=i}^n 1 = \sum_{i=1}^n (n - i + 1) = \sum_{i=1}^n (n + 1) - \sum_{i=1}^n i =$$

$$n(n + 1) - \frac{n(n + 1)}{2} = \frac{n(n + 1)}{2} \approx n^2$$

插入排序（Insertion Sort）



插入排序的核心思想：

逐步将未排序的元素插入到已排序的部分中，并通过移动元素来为新

元素腾出空间。

```

template<class Type>
void insertion_sort(Type *a, int n)
{
    Type key;
    for (int i = 1; i < n; i++){
        key=a[i];
        int j=i-1;
        while( j>=0 && a[j]>key ){
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
}

```

$$T(n) = c_1 n + c_2 (n-1) + c_3 (n-1) + c_4 \sum_{i=1}^{n-1} t_i + c_5 \sum_{i=1}^{n-1} (t_i - 1) + c_6 \sum_{i=1}^{n-1} (t_i - 1) + c_7 (n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=1}^{n-1} t_i + c_5 \sum_{i=1}^{n-1} (t_i - 1) + c_6 \sum_{i=1}^{n-1} (t_i - 1) + c_7(n-1)$$

- 在最好情况下, $t_i = 1$, for $1 \leq i < n$;

已排好序

$$T_{\min}(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1)$$

$$= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7) = O(n)$$

- 在最坏情况下, $t_i = i$, for $1 \leq i < n$;

倒序

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \quad \sum_{i=1}^{n-1} (i-1) = \frac{(n-1)(n-2)}{2}$$

$$\begin{aligned} T_{\max}(n) &= c_1 n + c_2(n-1) + c_3(n-1) + c_4 \frac{n(n-1)}{2} \\ &\quad + c_5 \frac{(n-1)(n-2)}{2} + c_6 \frac{(n-1)(n-2)}{2} + c_7(n-1) \\ &= O(n^2) \end{aligned}$$

是否最优算法?

最优算法

- 排序问题的计算时间下界为 $\Omega(n \log n)$ ，即任何算法解决该问题所需的最小时间复杂度。
- 计算时间复杂度为 $O(n \log n)$ 的排序算法，是最优算法。

最优算法

排序 算法	最好情况下时间 复杂度	平均情况下时间 复杂度	最坏情况下时间 复杂度	适用场景
冒泡 排序	$O(n)$	$O(n^2)$	$O(n^2)$	教学示例、小规模数据或几乎有序的数据。
选择 排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	教学示例、小规模数据或对交换次数有严格限制的场景。
插入 排序	$O(n)$	$O(n^2)$	$O(n^2)$	小规模数据、几乎有序的数据或作为其他排序算法的子过程（如快速排序优化）。
归并 排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	大规模数据、需要稳定排序的场景、外部排序（如磁盘文件排序）。
快速 排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	大规模数据、对内存使用有限制的场景（原地排序）。
堆排 序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	大规模数据、需要原地排序且对最坏情况时间复杂度有要求的场景。

学习要点:

- 掌握算法的计算复杂性概念。
- 掌握算法渐近复杂性的数学表述。
- 掌握算法分析的基本法则。
- 掌握算法分析中常用的复杂函数。

算法分析中常见的复杂性函数

函数阶名称	大 O 表示法
-------	-----------

常数阶	$O(1)$
-----	--------

对数阶	$O(\log n)$
-----	-------------

线性阶	$O(n)$
-----	--------

线性对数阶	$O(n \log n)$
-------	---------------

平方阶	$O(n^2)$
-----	----------

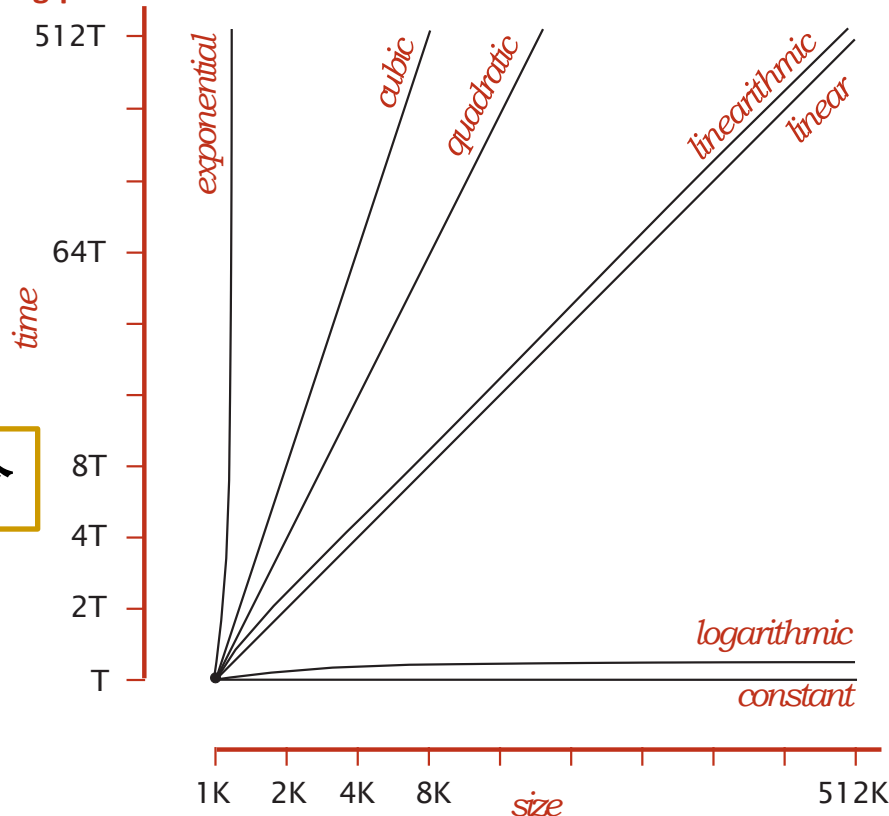
立方阶	$O(n^3)$
-----	----------

指数阶	$O(2^n)$
-----	----------

阶乘阶	$O(n!)$
-----	---------

多项式阶

log-log plot



多项式阶 $O(n^k)$ 只有在 $k > 1$ 时才高于线性对数阶 $O(n \log n)$; 若 $k = 1$, 则线性阶 $O(n)$ 更低。

算法分析中常见的复杂性函数

函数阶名称	大 O 表示法
常数阶	$O(1)$
对数阶	$O(\log n)$
线性阶	$O(n)$
线性对数阶	$O(n \log n)$
平方阶	$O(n^2)$
立方阶	$O(n^3)$
指数阶	$O(2^n)$
阶乘阶	$O(n!)$

可证明：对数函数的阶低于多项式函数($k \geq 1$)的阶

$$\text{当 } k \geq 1 \text{ 时, } \lim_{n \rightarrow \infty} \frac{\log n}{n^k} = \lim_{n \rightarrow \infty} \frac{\frac{d}{dn}(\log n)}{\frac{d}{dn}(n^k)} = 0$$

可证明：多项式函数的阶低于指数函数的阶

可证明：指数函数的阶严格低于阶乘函数的阶

1、算法概述

- ▶ *Why study algorithms?*
- ▶ *Course overview*
 - ▶ *1.1 Algorithm and program*
 - ▶ *1.2 Algorithm complexity analysis*
 - ▶ *1.3 NP hard theory of algorithms*

不可计算的问题

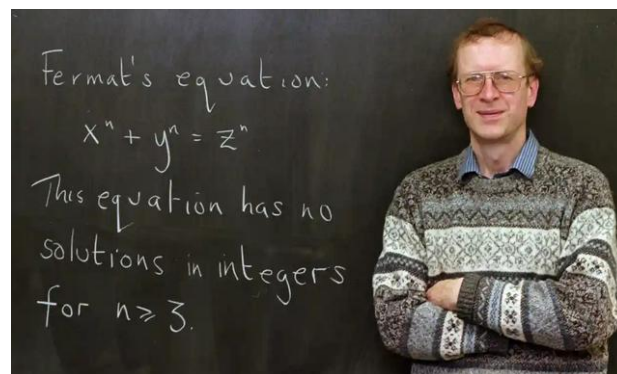
20世纪初，人们发现有许多问题无法找到解决的方法。于是开始怀疑，是否对这些问题来说，根本就不存在算法，即不可计算。

?

费马猜想：当整数 $n > 2$ 时，关于 x, y, z 的方程 $x^n + y^n = z^n$ 没有正整数解



费马 (Pierre de Fermat)
法国人，律师，业余数学家
(1601-1665)



- 1637年费马在阅读古希腊数学家丢番图的著作《算术》拉丁文译本时，在书右侧空白写下：将一个立方数分成两个立方数之和，或一个四次幂分成两个四次幂之和，或者一般地将一个高于二次的幂分成两个同次幂之和，这是不可能的。关于此，我确信我发现一种美妙的证法，可惜此处甚小，我无法写下证明过程。
- 英国数学家安德鲁·怀尔斯 (Andrew Wiles) 在1994年成功证明了费马大定理。

P类问题

■ P类问题(Polynomial Problem)

- 可以在多项式时间内通过算法求解的问题, 高效可解的问题
- 设计的算法时间复杂度小于多项式阶, 例如 $n^2, n^4, n(\log(n))$ 都是P时间的, 指数级别如 $2^n, n^n$ 就不是P时间
- P类问题, 如: 数值排序问题、查找问题

NP类问题

■ NP (Nondeterministic Polynomially, 非确定性多项式) 类问题:

- 可以在多项式时间内验证解的问题（尚未找到多项式时间算法）
- NP类问题是比P类问题更困难的问题
- 通常NP的时间复杂度都是指数阶
- 例如猜密码问题。验证密码是否正确容易，但暴力破解耗时极长。

例如，对方程 $x^{10} + 2x + 7 = 1035$ 佳，但是验证 很容易 $x = 2$

NP完全性理论

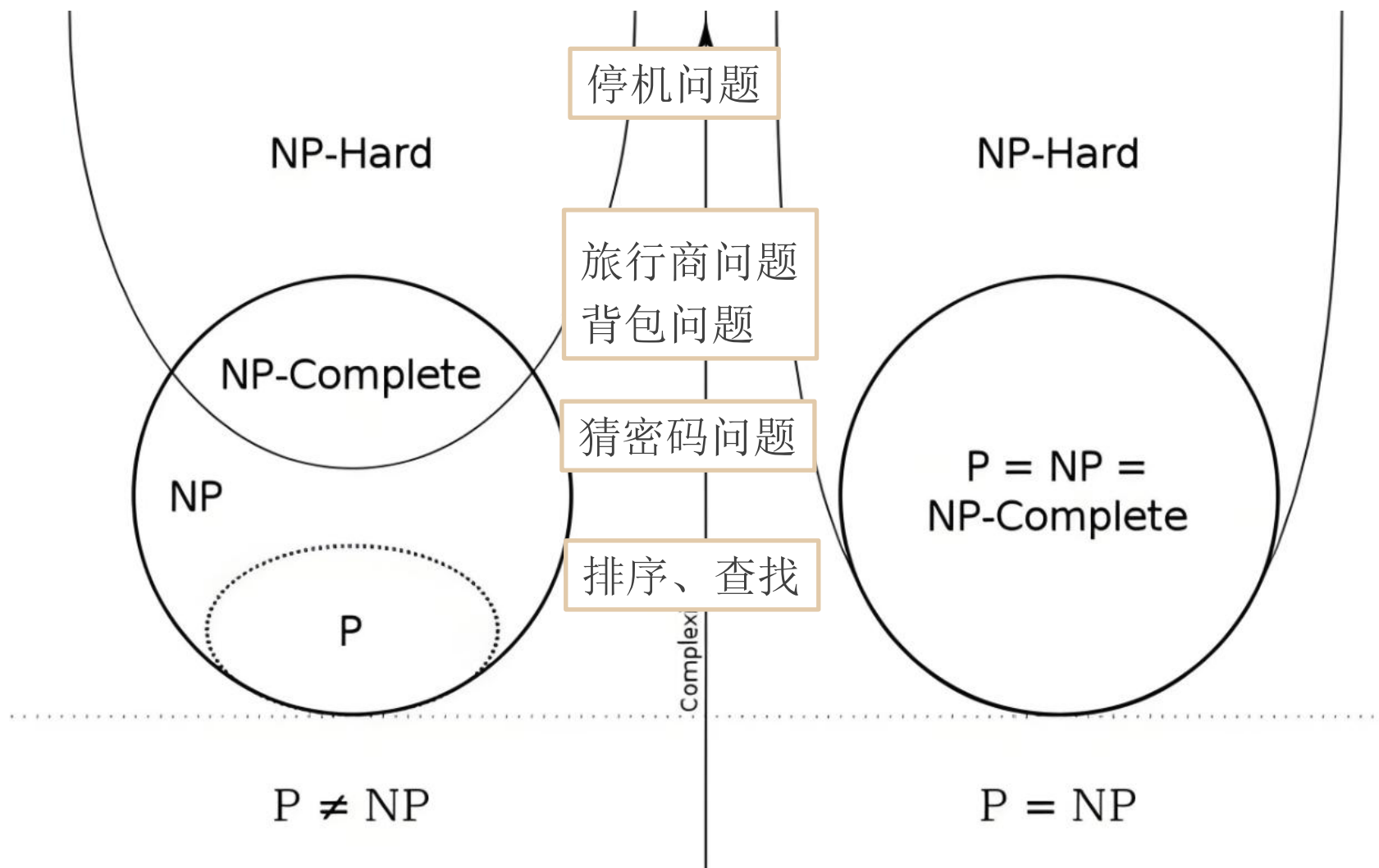
■ NP-Hard类问题:

- 其难度不低于任何的NP问题，但可能超出NP类的范围。如不可判定问题(停机问题)，解无法在多项式时间内验证

■ NP-Complete类问题

- NP中最难的问题，是NP与NP-Hard的交集

NP完全性理论



NP完全性理论

- **$P \subseteq NP$** : 所有P类问题均属于NP类问题。
- **P vs NP问题的意义**
 - 若 **$P=NP$** : 所有NP问题均可在多项式时间内解决。
 - 若 **$P \neq NP$** : 存在NP问题无法高效解决, NP-Complete问题属于此范畴。
- **现状**: P与NP的关系是计算机科学中最重要的未解问题之一。

作业

- 算法分析题1： 1-3,1-6,1-7
- 算法实现题1： 1-3,1-4

3.17前在canvas提交